Untitled

14 May 2026 23:44

HPC Assignment — Parallel BFS and Parallel DFS using OpenMP
This assignment implements:
Parallel Breadth First Search (BFS)
and
Parallel Depth First Search (DFS)
using:

- OpenMP
- graph traversal
- parallel programming
- synchronization
- adjacency matrix representation

Reference Style Based On:

1. What We Have To Do in This Assignment?
We have to:

1. create graph using adjacency matrix
2. perform BFS traversal
3. perform DFS traversal
4. apply OpenMP parallelism
5. process graph nodes using multiple threads
6. prevent race conditions using critical sections

Main Goal
To demonstrate:
parallel graph traversal using OpenMP
for:

- BFS
- DFS

2. What is BFS?
BFS =
Breadth First Search
Graph traversal algorithm.
It visits:
level by level

Example
Graph:

```
    0
   / \
  1   2
 / \   \
3   4   5
```

BFS Traversal from 0:
0 1 2 3 4 5

Why Called Breadth First?
Because:

- first all neighboring nodes visited
- then next level neighbors visited

3. What is DFS?

DFS =
Depth First Search
Graph traversal algorithm.
It visits:
depth wise
First it moves:

- deeply into graph
- then backtracks

Example
Same Graph:

```
    0
   / \
  1   2
 / \   \
3   4   5
```

DFS Traversal from 0:
0 1 3 4 2 5

Why Called Depth First?
Because:

- algorithm goes deeply into branch first
- then explores remaining branches

4. What is OpenMP?
OpenMP =
Open Multi-Processing
Used for:

- parallel programming in C/C++
- multicore CPU utilization
- thread-based execution

MOST IMPORTANT DIRECTIVES
Parallel For
#pragma omp parallel for
Converts loop into:
parallel loop.

Critical Section
#pragma omp critical
Allows:
only ONE thread at a time
to enter critical section.
Used for:

- synchronization
- preventing race conditions

5. Overall Workflow of Program
Input Graph
↓
Input Starting Vertex
↓
Perform BFS Traversal
↓
Process BFS Levels in Parallel
↓
Perform DFS Traversal

↓
Process DFS Stack Operations
↓
Display Traversals

6. Header Files
Code:
#include <iostream>
#include <vector>
#include <stack>
#include <omp.h>
Purpose

Header	Purpose
iostream	input/output
vector	dynamic arrays
stack	DFS stack
omp.h	OpenMP support

Namespace
using namespace std;
Avoids writing:
std::
again and again.

========== BFS ==========
7. BFS Function
Function:
void bfs(const vector<vector<int>> &graph, int start)
Purpose:
performs parallel BFS traversal.

Get Number of Vertices
Code:
int n = graph.size();
Stores:
number of graph vertices.

Visited Array
Code:
vector<bool> visited(n, false);
Purpose:
tracks visited nodes.
Initially:
all nodes unvisited.

Current and Next Level
Code:
vector<int> current_level, next_level;
Meaning

Vector	Purpose
current_level	nodes currently processing
next_level	nodes for next BFS level

Mark Starting Node
Code:
visited[start] = true;
current_level.push_back(start);
Starting node:

- marked visited
- added into BFS traversal

BFS Output
Code:
cout << \"\\nBFS Traversal: \";
Displays BFS traversal.

Main BFS Loop
Code:
while (!current_level.empty())
Loop continues until:
all graph nodes processed.

Clear Next Level
Code:
next_level.clear();
Why Needed?
Removes old nodes before storing new BFS level.

Print Current Level Nodes
Code:
for (int node : current_level)
Displays traversal order.

MOST IMPORTANT PART — Parallel BFS
Code:
#pragma omp parallel for
Converts loop into:
parallel execution.
Different threads process:
different nodes simultaneously.

Process Current Level Nodes
Code:
for (int i = 0; i < (int)current_level.size(); i++)
Each thread processes:
one node from current level.

Current Node
Code:
int u = current_level[i];
Stores:
current processing node.

Traverse Neighbors
Code:
for (int v = 0; v < n; v++)
Checks:
all adjacent vertices.

Check Edge Exists
Code:
if (graph[u][v] == 1)
Meaning:
edge exists between:
- u
- v

should_add Variable
Code:
bool should_add = false;
Purpose:
decides whether node should enter next level safely.

MOST IMPORTANT PARALLEL CONCEPT — Critical Section
Code:
#pragma omp critical
Only ONE thread enters critical section at a time.

Why Critical Section Needed?
Multiple threads may:
- access visited array simultaneously
- insert duplicate nodes

Without critical section:
race condition occurs.

What is Race Condition?
When multiple threads access shared data simultaneously causing:
- incorrect output
- duplicate operations
- inconsistent behavior

Visit Node Safely
Code:
if (!visited[v]) {
visited[v] = true;
should_add = true;
}
If node not visited:
- mark visited
- allow insertion into next level

Add Node to Next Level
Code:
next_level.push_back(v);
Stores node for future BFS processing.
Also protected using:
critical section.

Move to Next Level
Code:
current_level = next_level;
After current level finishes:
next level becomes current level.

Example BFS Execution
Suppose graph:

0 -- 1
| |
2 -- 3
Start Node:
0
Iteration 1:
Current Level = [0]
Visit:
- 1
- 2

Next Level:
[1,2]
Iteration 2:
Current Level:
[1,2]
Visit:
- 3

Final BFS:
0 1 2 3

How Parallelism Happens in BFS?
Suppose current level:
[1,2,3]
Thread 1:
process neighbors of 1
Thread 2:
process neighbors of 2
Thread 3:
process neighbors of 3
simultaneously.
This improves:
- execution speed
- CPU utilization

========== **DFS** ==========
8. DFS Function
Function:
void dfs(const vector<vector<int>> &graph, int start)
Purpose:
performs DFS traversal using stack.

Get Number of Vertices
Code:
int n = graph.size();
Stores:
number of vertices.

Visited Array
Code:
vector<bool> visited(n, false);
Tracks:
visited nodes.

Stack Creation
Code:
stack<int> st;
Purpose:

stores DFS traversal nodes.

Push Starting Node
Code:
st.push(start);
Adds starting node into stack.

DFS Output
Code:
cout << \"\\nDFS Traversal: \";
Displays DFS traversal.

Main DFS Loop
Code:
while (!st.empty())
Continues until:
stack becomes empty.

Current Node Variable
Code:
int node;
Stores:
current DFS node.

MOST IMPORTANT PART — Critical Section for Stack Access
Code:
#pragma omp critical
Protects:
shared stack operations.

Safe Stack Access
Code:
node = st.top();
st.pop();
Why Critical Needed?
Because:
multiple threads accessing same stack may cause:

- corruption
- invalid access
- race conditions

Visit Node
Code:
if (!visited[node])
Checks:
whether node already processed.

Mark Visited
Code:
visited[node] = true;
Marks node visited.

Display Node
Code:
cout << node << \" \";
Prints DFS traversal order.

Push Neighbors in Reverse Order
Code:
for (int i = n - 1; i >= 0; i--)
Traverses neighbors in reverse order.

Why Reverse Order Used?
Maintains:
correct DFS traversal sequence.

Check Edge Exists
Code:
if (graph[node][i] == 1 && !visited[i])
Checks:
- edge exists
- node not visited

Push Into Stack Safely
Code:
```
#pragma omp critical
{
   st.push(i);
}
```
Protects:
shared stack modification.

Example DFS Execution
Graph:
```
     0
    / \
   1   2
  / \   \
 3   4   5
```
DFS Traversal:
0 1 3 4 2 5

Important Reality About DFS Parallelism
DFS is difficult to parallelize efficiently because:
- traversal order highly dependent
- stack operations sequential in nature

In this implementation:
OpenMP mainly used for:
- synchronization
- safe shared stack handling

9. Adjacency Matrix Representation
Code:
vector<vector<int>> graph(n, vector<int>(n));
Creates:
2D adjacency matrix.

What is Adjacency Matrix?
Graph representation using matrix.

Example
0 1 1
1 0 1
1 1 0

Meaning:

- node 0 connected to 1 and 2
- node 1 connected to 0 and 2

Input Graph
Nested loops:
for (int i = 0; i < n; i++)
read adjacency matrix.

Starting Vertex
Code:
int start;
Stores:
starting traversal node.

10. Main Function
int main()
Program execution starts here.

BFS Function Call
Code:
bfs(graph, start);
Executes BFS traversal.

DFS Function Call
Code:
dfs(graph, start);
Executes DFS traversal.

11. Compilation Command
g++ -fopenmp graph.cpp -o graph
Run
./graph

Why -fopenmp Needed?
Enables:
OpenMP support.
Without it:
#pragma omp
ignored.

12. Time Complexity
Sequential BFS
O(V+E)
Where:

- V = vertices
- E = edges

Parallel BFS
Approximate:
O\left(\frac{V+E}{P}\right)
Where:

- P = processors/threads

Sequential DFS
O(V+E)

Important Reality
Actual performance depends on:

- synchronization overhead
- critical sections
- graph structure
- thread management

13. Advantages of Parallel Graph Traversal

Advantage	Explanation
Faster traversal	multicore utilization
Better CPU usage	simultaneous processing
Efficient large graphs	parallel node processing
Useful in HPC	graph analytics

14. Important Viva Questions
Q1. What is BFS?
Breadth First Search graph traversal algorithm.

Q2. What is DFS?
Depth First Search graph traversal algorithm.

Q3. Difference between BFS and DFS?
BFS traverses level-wise while DFS traverses depth-wise.

Q4. What is OpenMP?
API for parallel programming in C/C++.

Q5. What does #pragma omp parallel for do?
Parallelizes loop iterations.

Q6. What is race condition?
Incorrect behavior caused by simultaneous shared data access.

Q7. Why critical section used?
To prevent race conditions.

Q8. What is adjacency matrix?
2D matrix representation of graph.

Q9. Why visited array used?
To avoid revisiting nodes.

Q10. Why BFS suitable for parallelism?
Different nodes of same level can be processed simultaneously.

Q11. Why DFS difficult to parallelize?
Traversal order depends on stack operations.

Q12. Why stack used in DFS?
To maintain depth-wise traversal order.

Q13. What is current_level and next_level in BFS?
Used for level-wise traversal.

Q14. Why reverse order traversal used in DFS?

To maintain correct DFS sequence.

Q15. Why critical section used for stack?
To safely access shared stack.

15. Short Practical Explanation
This program implements Parallel BFS and DFS graph traversal using OpenMP. BFS processes graph nodes level-by-level using current_level and next_level vectors, where multiple nodes are processed simultaneously using #pragma omp parallel for. Critical sections are used to safely update shared data structures and prevent race conditions. DFS traversal uses a stack for depth-wise traversal, while critical sections protect shared stack access during push and pop operations. The assignment demonstrates graph traversal, synchronization, OpenMP parallelism, and thread-safe shared data handling.

From <https://chatgpt.com/c/6a05a79c-df14-8321-87b7-d41b1c4bf962>